A Lean-oriented core for the $Q_{a,b}$ package-coprimality proof Version 4: worker handoff and polished assumption boundary

Draft blueprint for review

June 28, 2026

Abstract

This note extracts a Lean-oriented core from the current proof of the $Q_{a,b}$ package-coprimality theorem. It is not intended to replace the full manuscript. Its purpose is to define a conditional formalization target with explicit theorem packs that are generous enough for a first Lean implementation, but narrow enough that later phases can open the packs and formalize their contents down to Mathlib and Lean-checkable certificates.

Version 4 incorporates the final polish review of the v3 package. The residual witness name is made Lean-friendly, the certificate interfaces no longer import the core proof, the lower-box hypothesis is passed explicitly to the lower finite eliminators, and the phase-1 polynomial semantics now distinguish primitive orientations, full imprimitive orientations, reciprocal package products, and orientation-level versus product-level sharing.

1 Purpose and scope

The full manuscript contains discovery scaffolding and auxiliary finiteness arguments that are not needed in the shortest Lean validation path. The core retained here is the following:

$$\begin{gathered}\text{package sharing} \Rightarrow \text{normalized counterexample},\\ D \le 175{,}394{,}637,\\ D \ge 6{,}816{,}242 \Rightarrow \text{upper unit-normalized contradiction},\\ D \le 6{,}816{,}241 \Rightarrow \text{admissible lower state},\\ \text{unit/both-nonunit/one-nonunit branch eliminations},\\ \text{residual one-nonunit envelope and terminal certificates}.\end{gathered}$$

The top-level Lean theorem will be conditional on named theorem packs. The computational eliminations should ultimately be discharged by Lean-checkable certificates rather than by trusting external loops.

The following ingredients are deliberately absent from the top-level core: ASZ-style unlikely-intersection boundedness, Beukers–Schlickewei S-unit finiteness, density-one isolation side results, and most sign/phase/Schur scaffolding. These may remain in the human manuscript as explanation or independent checks, but they are not part of the shortest Lean proof spine.

2 Constants and normalized counterexamples

We write

$$B_0 = 175{,}394{,}637, \qquad B_1 = 6{,}816{,}241, \qquad B_1^+ = 6{,}816{,}242,$$

$$R = 16{,}583, \qquad R^+ = 16{,}584, \qquad E_R = 2601.$$

Here B_0 is the global height–degree–Mahler bound, B_1^+ is the first upper-range value, B_1 is the lower-box cutoff, R^+ is the start of the large one-nonunit branch, R is the residual cutoff, and E_R is the residual degree bound.

Definition 1 (Normalized counterexample record)**.** *A Lean record* `NormCE` *contains natural numbers*

$$m < n, \qquad p < q, \qquad D, d, U, V_0,$$

with $m, p, d, U, V_0 > 0$*, together with*

$$D = \max\{m, n, p, q\}, \qquad \gcd(\gcd(m, n), \gcd(p, q)) = 1,$$

and a proof that the two normalized shapes are not the same up to swapping. Here d *is the degree of the selected primitive minimal polynomial, and* U, V_0 *are the absolute endpoint coefficients used in the finite branch analysis.*

Define

$$\begin{aligned} \mathrm{Unit}(ce) &:\Longleftrightarrow U = 1 \text{ and } V_0 = 1, \\ \mathrm{BothNonunit}(ce) &:\Longleftrightarrow 2 \le U \text{ and } 2 \le V_0, \\ \mathrm{OneNonunit}(ce) &:\Longleftrightarrow (U = 1 \text{ and } 2 \le V_0) \text{ or } (2 \le U \text{ and } V_0 = 1). \end{aligned}$$

Because $U, V_0 > 0$, the trichotomy

$$\mathrm{Unit}(ce) \vee \mathrm{BothNonunit}(ce) \vee \mathrm{OneNonunit}(ce)$$

is elementary arithmetic. It is proved directly in Lean and is not part of the local p-adic theorem pack.

3 Opaque selected counterexamples

The selected counterexample predicate is

$$\texttt{FromShare}(P, Q, ce).$$

For the first conditional formalization it is an opaque predicate. It must not be defined as `True`. Otherwise broad finite-elimination assumptions would apply to arbitrary dummy records and could make the development inconsistent.

In a later phase, `FromShare` should become a structure containing the actual selected orientations, an off-unit-circle common root, the primitive minimal polynomial, degree and endpoint-coefficient data, and divisibility by the selected package polynomials.

4 Package polynomial semantics

The first concrete replacement for opaque `PackageShare` must be careful about primitive versus imprimitive pairs. The coefficient formula below is for the primitive orientation polynomial only. For coprime positive A, B, define

$$Q_{A,B}^{\mathrm{prim}}(x) = \sum_{0 \le j < A} B(j+1)x^j + \sum_{A \le j \le A+B-2} A(A+B-j-1)x^j.$$

For a nonprimitive pair $a = gA$, $b = gB$, with $g = \gcd(a, b)$, the full orientation polynomial should use the manuscript convention

$$Q_{a,b}(x) = gQ^{\mathrm{prim}}_{A,B}(x^g).$$

The reciprocal package product is

$$P_{a,b}(x) = Q_{a,b}(x)Q_{b,a}(x).$$

Thus the Lean development should keep separate names:

`qPrimZ`, `qOrientZ`, `qPackageProdZ`, `OrientationShare`, `PackageShare`.

Product-level `PackageShare` is closest to the theorem statement. If the collision dictionary uses orientation-level sharing, it should prove a separate bridge from product sharing to an orientation share for some reciprocal-choice of orientations.

5 The theorem packs

5.1 Collision pack

```
structure CollisionPack where
  share_to_normCE :
    ∀{P Q : PosPair},
      P.unequal →Q.unequal →¬P.sameUnordered Q →
      PackageShare P Q →∃ce : NormCE, FromShare P Q ce
```

This pack should eventually be opened by formalizing the package polynomial layer, product-to-orientation sharing, the collision dictionary, removal of forced cyclotomic roots, reciprocal orientation handling, no-three compatibility, and primitive normalization.

5.2 Global height pack

```
structure GlobalHeightPack where
  absolute_bound :
    ∀{P Q : PosPair} {ce : NormCE},
      FromShare P Q ce →ce.D ≤175394637
```

It packages the height of a collision root, the rank-two torus/isolation degree bound, the Mahler/Smyth gap or replacement, and the exact constant arithmetic. It is the part of the proof that makes the infinite family finite.

5.3 Upper normalization and upper elimination

```
structure UpperNormalizationPack where
  upper_unit_normalized :
    ∀{P Q : PosPair} {ce : NormCE},
      FromShare P Q ce →6816242 ≤ce.D →ce.UnitCoeff

structure UpperRangePack where
  no_upper_unit_counterexample :
    ∀{P Q : PosPair} {ce : NormCE},
      FromShare P Q ce →ce.UnitCoeff →
      6816242 ≤ce.D →ce.D ≤175394637 →False
```

This split reflects the proof more faithfully than a single broad upper-range contradiction. Later, the second pack can be replaced by coverage certificates and terminal row exclusions.

5.4 Local lower-state extraction

```
structure LowerState where
  rowId : Nat

constant Encodes : NormCE →LowerState →Prop
constant AdmissibleLowerState : LowerState →Prop

structure LocalPacketPack where
  to_lower_state :
    ∀{P Q : PosPair} {ce : NormCE},
      FromShare P Q ce →ce.D ≤6816241 →
      ∃st : LowerState, Encodes ce st ∧AdmissibleLowerState st
```

In later phases, `LowerState` should expose primitive shapes, scales, extraction degrees, endpoint coefficients, slope signatures, packet occupancies, deficient-prime data, total-radical constraints, and row IDs.

5.5 Residual envelope

For the residual one-nonunit branch, the local pack also exposes reciprocal normalization and the exact residual envelope:

$$D \le 16{,}583, \qquad V_0 = 1, \qquad 2 \le U \le D, \qquad d \le 2601.$$

In Lean:

```
def ResidualEnvelope (ce : NormCE) : Prop :=
  ce.D ≤16583 ∧
  ce.Vabs = 1 ∧
  2 ≤ce.U ∧ce.U ≤ce.D ∧
  ce.d ≤2601

structure ResidualWitness (ce : NormCE) where
  ce' : NormCE
  st : ResidualState
  recnorm : ReciprocalNormalize ce ce'
  envelope : ResidualEnvelope ce'
  encodes : EncodesResidual ce' st
  admissible : AdmissibleResidualState st
```

This prevents the residual eliminator from hiding the reductions to $|V| = 1$, $2 \le U \le D$, and $d \le 2601$.

5.6 Finite lower eliminations

```
structure FiniteLowerPack where
  no_lower_unit :
    ∀{ce : NormCE} {st : LowerState},
      Encodes ce st →AdmissibleLowerState st →
```

```
    ce.D ≤lowerMax →ce.UnitCoeff →False

  no_both_nonunit :
    ∀{ce : NormCE} {st : LowerState},
      Encodes ce st →AdmissibleLowerState st →
      ce.D ≤lowerMax →ce.BothNonunit →False

  no_one_nonunit_large :
    ∀{ce : NormCE} {st : LowerState},
      Encodes ce st →AdmissibleLowerState st →ce.OneNonunit →
      16584 ≤ce.D →ce.D ≤6816241 →False
```

The explicit lower-box argument in the first two fields is deliberate. The state should also encode the lower box eventually, but the core proof already has the bound and passes it to make the finite-pack boundary auditable.

5.7 Residual finite elimination

```
structure FiniteResidualPack where
  no_residual_state :
    ∀{ce ce' : NormCE} {st : ResidualState},
      ReciprocalNormalize ce ce' →
      ResidualEnvelope ce' →
      EncodesResidual ce' st →
      AdmissibleResidualState st →
      False
```

The first concrete certificate milestone should target this pack.

6 Core conditional theorem

Theorem 1 (Core assembly theorem)**.** *Assume the seven theorem packs above. Then package sharing cannot occur between two distinct nontrivial reciprocal packages.*

Proof. Assume a package share between two distinct nontrivial packages. The collision pack gives a normalized counterexample *ce* with `FromShare P Q ce`. The global height pack gives $D \le B_0$.

If $D \ge B_1^+$, upper normalization gives unit endpoint coefficients, and the upper-range pack gives a contradiction.

Thus $D \le B_1$. The local packet pack gives an admissible lower state encoding *ce*. By the arithmetic endpoint trichotomy, *ce* is either unit, both-nonunit, or one-nonunit. The first two cases are eliminated by the lower finite pack using the explicit hypothesis $D \le B_1$.

In the one-nonunit case, if $D \ge R^+$, the large one-nonunit eliminator gives a contradiction. Otherwise $D \le R$. The local residual theorem supplies a reciprocally normalized residual witness satisfying $D \le R$, $V_0 = 1$, $2 \le U \le D$, and $d \le E_R$. The residual finite pack eliminates this state. Hence no normalized counterexample exists, and the original package share was impossible. □

7 Certificate strategy

The computational part should ultimately be Lean-checked by certificates, not trusted external loops. The certificate order should be:

(1) implement canonical primitive and imprimitive package polynomial constructors;

(2) formalize a good-reduction theorem for primitive common factors;

(3) check the residual terminal modular gcd certificates over $\mathbf{F}_{1009}$;

(4) add residual coverage certificates for the state generator and pair sieve;

(5) extend the same pattern to the both-nonunit, one-nonunit-large, and unit branches.

The schematic certificate interface now includes a terminal-row well-formedness predicate, an `exactGcdDegree` field, and a tag specifying whether the certificate checks divided package polynomials or undivided collision trinomials. For modular gcd certificates, the proof must specify which family is checked. If the certificate reports modular gcd degree 2 for collision trinomials, the degree 2 must be explicitly accounted for by the forced double root at $x = 1$.

8 Implementation phases

Phase 1. Compile the conditional scaffold. Build the Lake project, fix minor API drift, and keep the core theorem pack-parametric.

Phase 2. Define package polynomials and package sharing. Implement the primitive coefficient polynomial, the full imprimitive orientation polynomial, the reciprocal product, and product-level `PackageShare`.

Phase 3. Open the residual terminal certificates. Encode the small terminal residual rows and the modular gcd certificates over $\mathbf{F}_{1009}$.

Phase 4. Open residual coverage. Expand `ResidualState`, define row predicates, and verify coverage of the residual generator and pair sieve.

Phase 5. Open lower nonunit branches. Formalize the both-nonunit and large one-nonunit finite-state eliminations.

Phase 6. Open the unit branch. Formalize the full-Kummer terminal certificates and decide whether the Kummer-defect branch remains a narrow isolation axiom or is replaced by direct finite certificates.

Phase 7. Open the collision pack. Replace `FromShare` by real selected-root data and prove collision extraction.

Phase 8. Open the height and local theorem packs. Formalize or narrowly axiomatize the height–degree–Mahler argument, Borisov/Newton consequences, and Kummer extraction-degree control.

9 Open mathematical review questions

(1) Should the unit Kummer-defect branch keep a narrow global-isolation axiom for the 46 irreducible shapes, or should it be replaced by direct finite certificates?

(2) Should terminal modular gcd certificates be attached to divided package polynomials or to undivided collision trinomials?

(3) Should the upper-range final rows be excluded by the same-shape degree contradiction or by terminal modular certificates?

(4) What is the best eventual representation of `FromShare`: explicit field/root data, minimal-polynomial data, or divisibility-only data?

10 Acceptance criterion for the first Lean milestone

The first milestone is complete when:

(1) `lake build` succeeds;

(2) `package_coprimality_from_packs` compiles;

(3) `FromShare` is opaque or a genuine structure, not `True`;

(4) all broad temporary assumptions live only in `BroadAxioms.lean`;

(5) `Qab.lean` does not import `BroadAxioms.lean`;

(6) the README explicitly says the project is conditional on theorem packs;

(7) the core proof file remains a short assembly theorem.